Sprint 2 Deliverable

University Inventory Management System

Srinidh & Jithender

Sprint Number:	Sprint 2
Sprint Duration:	2 Weeks (Week 3 – Week 4)
Sprint Goal:	Set up the Department and Room management module — enabling Admins to add, edit, and delete departments and rooms with role-based access.
Team:	Srinidh, Jithender
Course:	Software Engineering Lab
Date:	March 2026

1. Sprint Plan & Backlog

1.1 Sprint Goal

Build the Department and Room Management module. Admins must be able to create, edit, and delete departments and rooms, view all rooms mapped to a department, and enforce validations (unique room names, mandatory fields). This sprint also introduces the application navigation shell (sidebar + header) that will be reused across all subsequent sprints (UC-002, UC-003).

1.2 Sprint Backlog

The following user stories and tasks are selected from the product backlog for Sprint 2.

ID	User Story / Task	Priority	Est. Hrs	Status
US-05	As an Admin, I want to add a new department so that rooms can be organised by department.	High	3	To Do
US-06	As an Admin, I want to edit or delete a department so that outdated entries are removed.	High	2	To Do
US-07	As an Admin, I want to add a new room with a location type and assign it to a department.	High	4	To Do
US-08	As an Admin, I want to view all rooms in a department in a sortable list.	Medium	3	To Do
US-09	As an Admin, I want to edit or delete a room so that the system stays accurate.	Medium	2	To Do
T-08	Design the Department and Room management UI pages (HTML/CSS/JS).	High	4	To Do
T-09	Implement REST API endpoints: POST/GET/PUT/DELETE for /departments and /rooms.	High	5	To Do
T-10	Add server-side and client-side validation (unique names, required fields).	High	3	To Do
T-11	Connect Room list page with dynamic department filter dropdown.	Medium	2	To Do
T-12	Unit test department and room CRUD operations.	Medium	3	To Do

1.3 Sprint Timeline

Day	Activity
Day 1–2	Set up navigation shell (sidebar/navbar) reused across the application
Day 3–4	Implement department CRUD API endpoints and MySQL queries
Day 5–6	Build Department management UI page; connect to backend API
Day 7–8	Implement room CRUD API endpoints; enforce FK constraint to DEPARTMENT
Day 9–10	Build Room management UI with department dropdown filter
Day 11–12	Add validation (unique names, empty fields); render error messages in UI
Day 13–14	Unit tests, bug fixes, sprint review and retrospective

1.4 Definition of Done

- Admin can add, edit, and delete departments without page reload errors.
- Admin can add, edit, and delete rooms, each linked to a valid department.
- Room list page filters correctly by department selection.
- Duplicate room names within the same department are rejected with a clear error message.
- All REST endpoints return appropriate HTTP status codes (200 / 201 / 400 / 404).
- Unit tests pass for all CRUD operations.
- Navigation shell renders correctly on Chrome and Firefox.
- Code is committed to the repository with meaningful commit messages.

1.5 Sprint Risks

Risk	Likelihood	Impact	Mitigation
Foreign key constraint errors when deleting a department that still has rooms.	Medium	Medium	Warn admin before deletion; block delete if child rooms exist.
Department dropdown not populating in real time on Room form.	Low	Low	Fetch departments via AJAX on page load; cache result.
UI inconsistency across browsers.	Low	Low	Test on Chrome and Firefox; use a CSS reset.

2. Module Design

2.1 Use Cases Addressed

Use Case	Description
UC-002	Admin adds, edits, or deletes a department.
UC-003	Admin adds, edits, or deletes a room and assigns it to a department.

2.2 API Endpoints

Method	Endpoint	Description
GET	/api/departments	List all departments.
POST	/api/departments	Create a new department.
PUT	/api/departments/:id	Update department name or description.
DELETE	/api/departments/:id	Delete department (blocked if rooms exist).
GET	/api/rooms	List all rooms; optional ?dept_id= filter.
POST	/api/rooms	Create a new room linked to a department.
PUT	/api/rooms/:id	Update room details.
DELETE	/api/rooms/:id	Delete room (blocked if items are assigned).

2.3 Sample Node.js Route — POST /api/departments

```
router.post('/departments', authMiddleware, adminOnly, async (req, res) => {
const { dept_name, description } = req.body;
if (!dept_name)
return res.status(400).json({ error: 'dept_name is required' });
const [result] = await db.query(
'INSERT INTO DEPARTMENT (dept_name, description) VALUES (?, ?)',
[dept_name, description]
);
res.status(201).json({ dept_id: result.insertId, dept_name });
});
```

2.4 Database — DEPARTMENT Table (added this sprint)

```
CREATE TABLE DEPARTMENT (
dept_id INT PRIMARY KEY AUTO_INCREMENT,
dept_name VARCHAR(100) NOT NULL UNIQUE,
description TEXT,
created_at DATETIME DEFAULT CURRENT_TIMESTAMP
);
-- Link ROOM to DEPARTMENT:
ALTER TABLE ROOM ADD COLUMN dept_id INT,
ADD FOREIGN KEY (dept_id) REFERENCES DEPARTMENT(dept_id);
```

3. Unit Test Summary

Test ID	Description
UT-2-01	POST /departments with valid data returns 201 and correct response body.
UT-2-02	POST /departments with a duplicate name returns 409 Conflict.

UT-2-03	DELETE /departments/:id when rooms exist returns 400 Bad Request.
UT-2-04	GET /rooms?dept_id=X returns only rooms belonging to that department.
UT-2-05	PUT /rooms/:id updates room_name correctly in the database.

4. Sprint Review & Retrospective

4.1 Completed Stories

- US-05 — Admin can add/edit/delete departments.
- US-06 — Duplicate department names blocked.
- US-07, US-08, US-09 — Room CRUD with department filter.
- T-08 through T-12 — UI, API, validation, and unit tests.

4.2 What Went Well

- CRUD pattern established cleanly — reusable for Inventory and Verification sprints.
- Navigation shell built once; saves effort across remaining 3 sprints.

4.3 What to Improve

- Add pagination early — full-fetch of large room lists is slow.
- Document API contracts in a shared spec before coding begins.